

RunSoC 2.0

Expert Talks

Semi-Structured Interview Protocol and Summary of Results

1. Purpose and Scope

This document defines the semi-structured expert-talk protocol used for RunSoC 2.0 and consolidates the key observations from the interview results. The interview structure focuses on heterogeneous MPSoCs, semiconductor design partitions, affinity-based allocation of automotive runnables/tasks, and practical constraints from mixed-criticality high-performance computing platforms.

The protocol is designed to complement the rapid literature review by eliciting practitioner knowledge on scheduling criteria, safety constraints, evaluation KPIs, bottlenecks, and implementation risks.

2. Interview Structure

The interview uses a funnel structure: a screening question determines whether the participant currently uses or plans to use heterogeneous MPSoCs. The follow-up questions then branch into heterogeneous-platform adoption or homogeneous-platform rationale. A common general block is asked independent of the branch.

2.1 Screening Question

No.	Interview question	Purpose / construct
S1	Do you use heterogeneous MPSoCs in the current project, or do you plan to use them in future projects?	Classifies the interview path and establishes platform context.

2.2 Branch A: Current or Planned Use of Heterogeneous MPSoCs

No.	Interview question	Purpose / construct
A1	What are the reasons for relying on, or transitioning to, heterogeneous MPSoCs?	Adoption drivers and expected architectural benefits.
A2	Which core affinities do you consider for these architectures, and how do you characterize them?	Affinity dimensions such as safety, real-time behavior, memory locality, compute type, and performance.
A3	Based on which criteria do you schedule and allocate tasks to heterogeneous MPSoC partitions?	Allocation criteria and decision logic.
A4	Which KPIs do you use to evaluate scheduling and allocation concepts?	Evaluation measures used in practice.
A5	Which benefits, risks, and challenges do you see in the use of heterogeneous MPSoCs?	Perceived trade-offs and engineering barriers.

2.3 Branch B: No Current or Planned Use of Heterogeneous MPSoCs

No.	Interview question	Purpose / construct
B1	What is the reason for staying with homogeneous MPSoCs in your domain?	Rationale for homogeneous architectures.
B2	What are the current bottlenecks?	Existing architectural or scheduling limitations.
B3	Do you see any potential improvements from introducing a heterogeneous hardware platform?	Potential value of heterogeneity despite current non-adoption.

2.4 General Questions

No.	Interview question	Purpose / construct
G1	How does a high ASIL requirement constrain your choice of scheduling algorithms?	Safety-driven restrictions on scheduling and allocation.
G2	How do you ensure ASIL compliance in scheduling and allocation strategies?	Safety assurance and partitioning mechanisms.
G3	Does a high ASIL always imply a low-latency requirement in your domain?	Relationship between safety integrity and timing criticality.
G4	Would you rate event-based tasks with real-time behavior as supportive for relieving workload?	Potential benefit of event-driven execution.
G5	Do you use mechanisms for load balancing across available cores, and how are they implemented?	Practical load-balancing approaches.
G6	Do you use fault-tolerance mechanisms against scheduling problems such as priority inversion?	Robustness and scheduling failure mitigation.

2.5 Suggested Probes and Clarification Questions

Topic	Probe question
Memory contention and bandwidth	Is memory contention or interconnect bandwidth currently a bottleneck in your application?
Data locality	How do you handle data locality when allocating tasks to cores?
Worst-case execution time	How do you determine WCET for relevant runnables/tasks?
Interrupt modeling	Must each interrupt be handled as a single task, or can interrupts be aggregated or modeled differently?

3. Summary of Results

The following synthesis consolidates the observed statements from the expert talk into reusable findings for the RunSoC 2.0 research method and modeling activities.

3.1 Affinity and Allocation Logic

- Safety-critical ASIL tasks are assigned to ASIL-capable safety cores.
- For homogeneous platforms, affinity is mostly hardware-defined; no theoretical optimization-based affinity model is currently applied.

- Data locality remains an important practical allocation consideration, particularly when load balancing across cores.

3.2 Benefits, Risks, and Challenges

Dimension	Observation	Implication for RunSoC 2.0
Complexity	Heterogeneous MPSoCs increase system and data-synchronization complexity.	The framework should make allocation rationale explicit and auditable.
Readability	Allocation and scheduling decisions can reduce system readability.	The output should provide interpretable mappings, not only optimized assignments.
Real-time capability	Additional complexity can reduce real-time predictability.	Timing constraints and response-time KPIs should remain first-class model elements.

3.3 Evaluation KPIs

KPI	Expert statement	Interpretation
Response time	Timing from call to response is a central KPI.	Response time can represent several lower-level effects, including compute and communication delays.
Stack usage	Critical: a full stack prevents further processing.	Stack constraints should be modeled explicitly when grouping SWCs or runnables.
RAM usage	RAM should be minimized to preserve headroom for future features and additional applications.	Memory consumption is a resource objective, not only a feasibility constraint.
CPU utilization and communication latency	Not considered separately when their direct influence is already reflected in response time.	RunSoC 2.0 may model them as explanatory variables or secondary KPIs, depending on validation needs.

3.4 ASIL and Safety

- In the discussed context, the safety allocation is not considered highly complex: safety tasks are assigned to safety-capable cores.
- A high ASIL requirement does not necessarily imply a low-latency requirement.
- DTOS safety scheduling is not yet defined.
- Priority inversion and related scheduling failures were not considered a primary concern in the discussed setup.

3.5 Load Balancing and Data Locality

The target is to keep workloads as balanced as possible across available cores while maintaining data locality. This creates a trade-off between load distribution and communication/memory efficiency.

3.6 Expert Suggestions for RunSoC 2.0

- Use SWC grouping to minimize stack space, although this is not equivalent to full scheduling.
- Randomness in executions should be considered to simulate more realistic execution behavior.